Supplementary Material

Why Should They Expand? Computational Growth, Resource Sufficiency, and Conditional Interstellar Expansion

Hyeonjun Joo

12 October 2026

S1 Numerical checks and concavity certificates

Corner classifications are determined analytically, without optimizing budget shares. A separate implementation also checks every stored optimizer output. For budget-share vectors v in the unit simplex, let

$$G(v) = \max_j [\nabla U(v)]_j - \nabla U(v) \cdot v.$$

For any other feasible share vector v', concavity gives $U(v') - U(v) \le \nabla U(v) \cdot (v' - v) \le G(v)$, since $\nabla U(v) \cdot v'$ is a convex combination of gradient entries. This proves the objective-gap bound. The reported normalized certificate is $\max\{0, G(v)\}/(1 + |U(v)|)$; truncation removes only negative floating-point round-off. It is not an exact-arithmetic optimizer proof.

The audit reconstructs the objective and gradient directly from the frozen parameters and stored budget shares. It does not import the original model. All 14,336 stored solutions are feasible and pass the 2×10^{-6} normalized-gap tolerance. The largest observed normalized gap is 1.32370×10^{-6}, and the largest absolute objective reconstruction difference is 3.55271×10^{-15}. All exact corner classifications match the archived flags. At every finite positive productivity threshold, the code checks $a = a_{\text{crit}}(1 - 10^{-6})$ and $a = a_{\text{crit}}(1 + 10^{-6})$, for 11,002 tests in total.

A separate scalar optimizer checks the pure-computation allocation at 101 budgets, with maximum objective discrepancy 1.81899×10^{-12}. The two-period construction is checked at 201 external-yield settings against a joint optimization in (x_1, z_2, x_2) with explicit second-period budget feasibility. Two initial conditions are used for each joint optimization. The maximum absolute objective difference is 1.49036×10^{-12}. The reproduced transition is $g_X = 0.4091479213$; at $g_X = 2$, $x_1 = 3.7559168566$ and $B_2 = 9.7606503419$. The script also rechecks all four support paths, the finite-depth lemma, the nonconcave negative control and the fixed/random-origin silence probabilities. It does not rerun the original Monte Carlo population simulations or the removed settlement benchmark.

S2 Generator provenance and frozen-input reproduction

The original environment record gives Python 3.13.5, NumPy 2.3.5 and SciPy 1.17.0. The normalized call is

```
u = scipy.stats.qmc.Sobol(
    d=14, scramble=True, seed=20260905
).random_base2(m=10)
```

The original source writes the same dimensions and power through its variables `len(NAMES)` and `power`. The effective bit depth is 30, with no optimization, skipping or thinning. The archive stores both the unit points and transformed parameter rows. The reference file's SHA-256 is

`ce45ecd0435f2abf834445957db369d1d4a5bc28e9c4e7b686561df50a14194b`

Array hashes use contiguous row-major little-endian float64 bytes and are recorded in `results/archive_audit.json`.

Table S1: Same-version generator-interface comparison. All calls use SciPy 1.17.0, $d = 14$, $m = 10$, scrambling and the same integer seed. "Same points" means bit-for-bit equality with the archived unit array.

Interface	Same points	Reference	Perfect substitution
`seed=20260905`	Yes	219	527
`rng=20260905`	No	208	530

The `seed` call reproduces the unit points exactly. The transformed parameters differ in some last bits, with maximum relative difference 2.21773×10^{-16}, without any classification change. Both keyword paths instantiate NumPy Generator/PCG64 objects in the environment tested; it would therefore be inaccurate to describe the observed difference simply as RandomState versus Generator. With `seed=20260905`, the archived unit points are reproduced bit for bit under SciPy 1.17.0 and 1.18.1; `rng=20260905` yields a different scramble, with 208 instantaneous and 530 perfect-substitution inward corners. The frozen arrays, not the seed alone, define the reference experiment.

Eight further scrambles for each of log-uniform and linear-uniform marginals are regenerated and agree with the archived measure-sensitivity results. These checks assess the selected mathematical design, not an extraterrestrial sampling distribution.

S3 New mathematical examples and executable outputs

Growing budgets and persistence

For the one-direction example $r = 0.1$, $S = 0.001$, $d_0 = 2$ and fixed external price $p = 1$, the required logarithmic price is $H + A/a$. The equation $H(B) = 1$ has the numerical solution $B_\dagger \simeq 189.490600549$. At $B \geq B_\dagger$ no finite productivity can meet the corner test with these fixed coefficients. Table S2 also checks constructive paths satisfying the exact condition. In the price-growth path, $p(B) = 2H(B)$ and $a(B) = B + A(B)/H(B)$. In the residual-decay path, $p = 1$, $S(B) = 1/[2(1 + B)\ln(1 + B)]$, and $a(B) = B + 2A(B)$. All points are deliberately constructed examples.

Table S2: Growth-path checks. The last two columns report required price divided by actual price for the two constructive paths; values below one pass the corner test.

B	H (fixed S)	Required p at $a = 100$	Price-growth path	Residual-decay path
10	0.026377	0.037597	0.904830	0.616390
100	0.466127	0.569147	0.590504	0.585521
1000	6.915664	7.936684	0.507275	0.583503

Curvature trichotomy

For $B = 10$, $r = 0.1$, $S = 0.001$, $d_0 = 2$ and $p = 1$, the computed thresholds are listed in Table S3. Global scalar optimizations use

$$U(x) = F_\eta\big(1 + a\ln(1 + 10 - x) + 0.1x\big) + 0.001(1 - e^{-x}), \qquad 0 \leq x \leq 10,$$

which is concave for $\eta \geq 0$. They compare the derivative-based corner classification with the actual optimized objective; a dense grid additionally challenges each scalar result. At $\eta = 2$, the code finds outward solutions for $a = 1, 20$ and inward solutions for $a = 2, 10$.

Table S3: Curvature examples. The numbers are generated by `validate_additions.py`.

η	Inward productivity set
0	$[1.111000000, \infty)$
0.5	$[1.123834330, \infty)$
1	$[1.152396590, \infty)$
2	$[1.421896643, 12.720520655]$

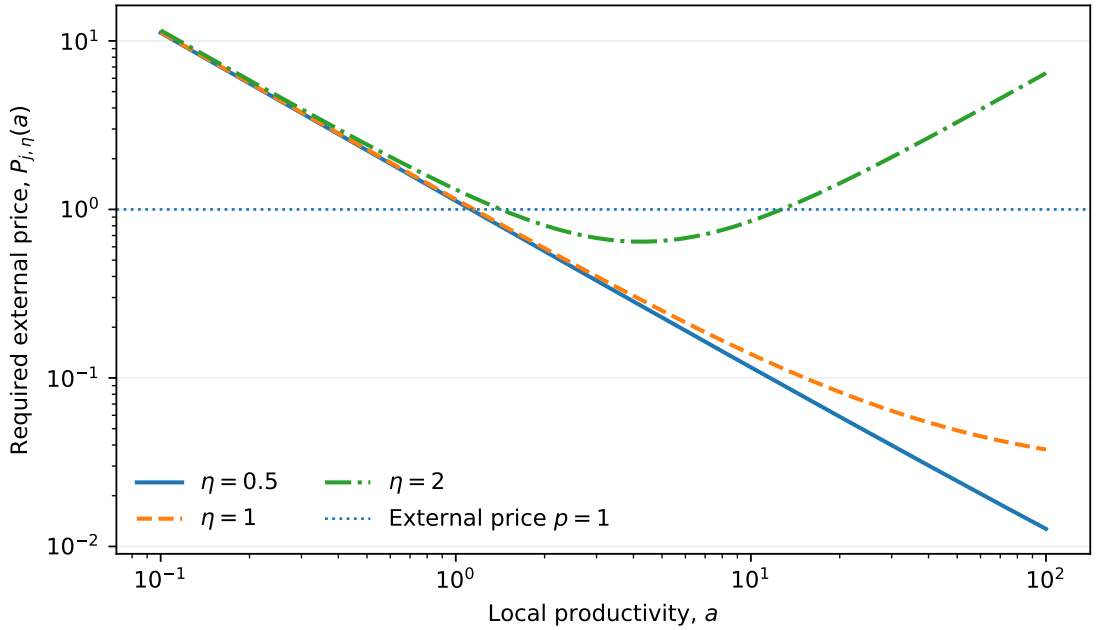


Figure S1: Required external price as a function of productivity in the curvature example. Inwardness requires the curve to be at or below the horizontal price. The curves are evaluations of the analytical formula, not fitted lines.

Production, recursion and support horizon

The production check compares $10\ln(1 + B - x) + x$ with $10\ln(1 + B - x) + \ln(1 + x)$ at $B = 8, 10, 100$, using both derived solutions and scalar optimization. The shared inward boundary is $B = 9$, but the interior allocations differ. The generalized production lemma is additionally checked with logarithmic, linear and shifted square-root local production.

The recursion identity is checked at $b = 3$, $d = 4$ and $c = 0.7$: $W_d = 121$ and $W_{d+1} = 364$; the cost ratio is 3.008264463, while the logarithmic value increment is 1.093543064. The latter approaches $\ln b$, not a multiplicative value gain. The fixed-memory support extension gives $W = 255$ and $255/256$ at $t = 50$, and $W = 511$ and $511/256$ at $t = 60$.

Static and dynamic choices can reverse in either direction

Both cases use $B_1 = 10$, $p_1 = p_2 = r_1 = r_2 = 1$, $C_{01} = C_{02} = 1$, $a_2 = 100$ and $\beta = 0.9$. Case A has $a_1 = 100$, $B_{20} = 1$, $g_L = 0.2$, $g_X = 2$; case B has $a_1 = 5$, $B_{20} = 0$, $g_L = 0.1$, $g_X = 0$. In case

B, the present-only maximizer is $x = 6$ because $z_1^* = a_1 p_1 / r_1 - 1 = 4$. At the inward corner, its continuation budget is $B_2 = 1$, so

$$D_{\text{static}} = \frac{1 - 5/11}{2 + 5 \ln 11}, \qquad D_{\text{future}} = -0.09 \frac{100}{2(2 + 100 \ln 2)}.$$

The sum is negative, proving the inward two-period optimum by concavity. The opposite signs in case A show why a static corner is not sufficient either. The complete output is `results/v73_checks.json`.

Table S4: New checks of the two directional reversals. The terms sum to the derivative of the full two-period objective at $x = 0$.

Case	Static term	Future term	Total	Static x^*	Dynamic x^*
A	-0.03346261	+0.28799092	+0.25452830	0	3.75591686
B	+0.03899035	-0.06310058	-0.02411023	6	0.00000000

The new script reconstructs the continuation formula and checks the endpoint or derivative-root optimum against a bounded scalar optimizer including both endpoints. A separate SLSQP formulation optimizes (x_1, z_2, x_2) jointly, with an explicit second-period budget constraint and three feasible initial conditions in each case. A grid also checks the objective and the monotonicity of its derivative. These are numerical implementation checks, not replacements for the concavity proof.

A lower bound for general concave local production

Corollary 7.1 requires differentiability at $B > 0$, not at the origin, so it includes $\ell(z) = \sqrt{z}$. The ratios are

$$\frac{\ell(B)}{\ell'(B)} = \begin{cases} 2B, & \ell(z) = \sqrt{z}, \\ e^B - 1, & \ell(z) = 1 - e^{-z}, \\ B, & \ell(z) = z. \end{cases}$$

All meet the lower bound B. The code evaluates the functions and derivatives separately and checks these identities and the supporting-line inequality. Table S5 uses $S_j = 0.001$.

Table S5: Residual-floor checks for three concave production functions. The final column is the general lower bound, not an approximation to the floor.

$\ell(z)$	B	$\ell(B)/\ell'(B)$	H_j^ℓ	$S_j B$
$\sqrt{z}$	1	2.000000	0.002000000	0.001
$\sqrt{z}$	2	4.000000	0.004000000	0.002
$\sqrt{z}$	10	20.000000	0.020000000	0.010
$1 - e^{-z}$	1	1.718282	0.001718282	0.001
$1 - e^{-z}$	2	6.389056	0.006389056	0.002
$1 - e^{-z}$	10	22025.465795	22.025465795	0.010
z	1	1.000000	0.001000000	0.001
z	2	2.000000	0.002000000	0.002
z	10	10.000000	0.010000000	0.010

Reproduction commands and output mapping

From the package root, run:

```
sh reproduce.sh
```

The new numerical checks run through `code/validate_v73.py` and write `results/v73_checks.json` and their saved grids. The exact phase transformations, arrow coordinates, and updated Figure 3 are produced by `code/make_v73_phase.py`. The archive audit is rerun in `results/v73_archive_recheck/`, without overwriting its retained outputs. Figures 1, 2 and 4 and the original numerical tables are preserved byte for byte. The source-edit and export scripts synchronize the modular and expanded sources, and the build regenerates all three PDFs. The release check records source, hash and compilation checks in `results/v73_release_checks.json`.

S4 Scope of the source revision

The complete revised article, the curvature and persistence propositions, the production lemma, and the production-floor corollary with full proofs are in `manuscript/body.tex`. The human-readable single-file LaTeX release expands the manuscript sections, tables and bibliography. The package provides the final manuscript, mathematical proofs, frozen inputs, numerical outputs and executable reproduction code.